

FlowForge AI — UI Testing Final Report

1. Overview

This document serves as the formal UI Testing report and Execution Proof for the **FlowForge AI** web application. We have implemented an automated UI testing suite using **Selenium WebDriver** in Java, managed via Maven, and orchestrated using the **TestNG** framework.

The objective of these automated tests is to validate critical user flows, page navigation, form inputs, and ensure that the interactive elements of the SPA (Single Page Application) frontend communicate properly with the backend and render accurately.

2. Test Automation Architecture & Framework

The automated UI testing framework was built inside the `backend` module leveraging the following stack:

- **Selenium WebDriver (v4.25+)**: For programmatic browser interaction.
- **Java 17+**: Test implementation language.
- **TestNG**: For test execution, assertions, and parameterized test lifecycle hooks (`@BeforeMethod`, `@AfterMethod`, `@Test`).
- **WebDriverManager**: Automated driver binary management.

Design Pattern: Page Object Model (POM)

To ensure maintainability and reduce code duplication, we adhered to the **Page Object Model**.

- **BaseTest.java**: Handles WebDriver initialization, implicit waits, window maximization, headless mode configurations, and automated teardown. It also implements an automatic screenshot capture mechanism on test failure.
- **LoginPage.java**: Encapsulates the UI locators (email input, password input, submit button) and interactions for the authentication flow.
- **DashboardPage.java**: Encapsulates interactions for the main dashboard post-login, including sidebar navigation to the Projects page.

3. Test Scenarios Implemented

The test suite validates the following core functionalities:

A. Authentication & Form Input Validation (`LoginTests.java`)

- **testLoginPageLoads()**: Navigates to the base URL and verifies that the login form (email, password inputs, and submit button) renders correctly.
- **testSuccessfulLogin()**:
 - Validates form input capabilities by injecting credentials (`test_hash@flowforge.ai / password`).
 - Simulates the button click to trigger the REST API authentication request.
 - Verifies that successful authentication properly triggers frontend routing (React Router) to redirect the user to the `/dashboard` route.
- **testPageTitle()**: Ensures the browser tab title is correctly set to "FlowForge".

B. Navigation & Routing Validation (`ProjectTests.java`)

- **loginFirst() (Setup Hook)**: Uses existing Page Objects to perform a prerequisite login before executing protected route tests.
- **testProjectsPageLoads()**:
 - Uses the Sidebar navigation link to route from the Dashboard to the Projects Hub.
 - Asserts that the application successfully transitioned to the Projects page without reloading the entire window (validating the SPA behavior).
- **testNavigateToDashboard()**: Validates direct URL navigation to the `/dashboard` route for an authenticated session.

C. Task Board Validations (`TaskTests.java`)

- **Kanban Board Navigation**: Verifies that the Tasks Board routes correctly and that columns (Backlog, In Progress, AI Review, Completed) render properly for the authenticated user.
- **AI & Team Views**: Tests the loading and routing to the Team and AI Assistant modules.

4. Execution Proof (Screenshots)

Below is the execution proof for each individual Selenium UI testcase. The testing framework captured these screenshots upon completion of each test method to verify that the target views rendered appropriately.

1. testPageTitle

2. testLoginPageLoads

3. testSuccessfulLogin

4. testNavigateToDashboard

5. testProjectsPageLoads

6. testTasksPageLoads

7. testTeamPageLoads

8. testAIPageLoads

5. Execution Proof (Terminal Output)

The automated test suite executed successfully against the local environment (Vite frontend + Spring Boot backend + TestNG). Below is the terminal output from the Maven Test phase:


```
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 30.81 s -- in TestSuite
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

6. Conclusion

The implementation of Selenium WebDriver automated tests successfully validates the primary user journeys—specifically login mechanisms, form interactions, and dynamic React routing. The use of the Page Object Model ensures that the testing suite is robust, maintainable, and easily extensible.